1 - Bernardo el vendedor tiene que dar un vuelto. Esto requiere rápidos cálculos mentales para dar el vuelto con la mínima cantidad de monedas posible. Más importante aún, esto requiere gran destreza manual para manipular las monedas, y ese no es el fuerte de Bernardo. Existe una lista de N denominaciones de monedas, de a1 centavos, a2 centavos, ..., an-1 centavos y an centavos.Bernardo posee un suministro ilimitado de monedas de cada una de estas denominaciones. Bernardo utilizó lo que aprendió estudiando robótica para programar y construir un robot automático de entrega de vuelto que lo ayude en su negocio. Programó al robot utilizando un algoritmo denominado goloso, avaro o voraz (se les llama greedy en inglés). En el algoritmo, se consideran las denominaciones disponibles ai, y el monto restante de V centavos que falta entregar. Se elige en cada paso la más grande posible ak tal que ak <= V, es decir sin pasarse del vuelto restante por entregar V. Se agrega una moneda de esa denominación ak a la solución, y se resta su valor al vuelto restante V, es decir se realiza la asignación V <- V - ak. El algoritmo continúa hasta que sea V = 0.

Tu tarea consiste en ayudar a Bernardo a programar un programa que le indique si su programa es correcto, y si no lo es,le indique el menor posible de V para el cual su programa no utiliza la mínima cantidad posible de monedas.

Especificación del programa:

- Parámetros:
    1. a: Arreglo de N enteros con las denominaciones de monedas ai.
- Valor de retorno:
    1. Un único entero positivo que indique el mínimo valor de V tal que el algoritmo de Bernardo falla (es decir,no entrega la mínima cantidad de monedas posible para dar un vuelto V). Si para las denominaciones indicadas el algoritmo es correcto, se debe retornar -1.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 4, a = [1, 5, 11, 20]
    - Salida: 15 *(Ya que V = 15 es el primer fallo del algoritmo de Bernardo: usará cinco monedas formando 15 = 11 + 1 + 1 + 1 + 1, pero se puede hacer 15 = 5 + 5 + 5 usando solamente tres monedas).*
- Ejemplo 2:
    - Entrada: N = 4, a = [1, 5, 15, 30]
    - Salida: -1

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 12, a = [1, 6, 14, 29, 58, 115, 230, 460, 920, 1840, 3680, 7360]
- Formato de respuesta esperado en el sistema: Un número

2 - En este desafío nos adentraremos en el mundo de los autómatas celulares y la evolución artificial. Imagina un arreglo unidimensional de N celdas, donde cada celda puede tener un estado de 1 (activa) o 0 (inactiva).

El sistema evoluciona a lo largo de E ciclos de tiempo. Las reglas del proceso son las siguientes:

1. Inicialización algorítmica: El arreglo inicial de tamaño N se genera mediante una condición matemática. Para cada posición i (desde 0 hasta N - 1), el valor inicial es 1 si la condición (i * 7 + 3) % 5 == 0 se cumple, y 0 en caso contrario.
2. Ciclos de evolución: Se deben simular E ciclos consecutivos. En cada ciclo, cada celda se actualiza basándose en sus vecinos inmediatos (su valor anterior, el de su izquierda y el de su derecha, utilizando condiciones de frontera circulares: el vecino a la izquierda de la celda 0 es la celda N - 1, y el vecino a la derecha de la celda N - 1 es la celda 0).
    - La regla de transición para el siguiente estado de la celda i es: si la suma de los valores de la celda i y sus dos vecinos (izquierda y derecha) en el ciclo anterior es un número par, la celda pasa a ser 1; si la suma es impar, pasa a ser 0.

Tu tarea es escribir un programa que, dados los valores de N y E, ejecute la carga inicial y los E ciclos de evolución descritos, y finalmente retorne la suma total de todos los valores del arreglo resultante.

Especificación del programa:

- Parámetros:
    1. N: Entero que indica la cantidad de celdas (tamaño del arreglo).
    2. E: Entero que indica la cantidad de ciclos de evolución a simular.
- Valor de retorno:
    1. Un número entero que representa la suma total de los valores de todas las celdas del arreglo tras completar los E ciclos.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 10, E = 3
    - Salida: 4

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 300, E = 72
- Formato de respuesta esperado en el sistema: Un número

3 - En este desafío nos adentraremos en el mundo de los autómatas celulares bidimensionales. Imagina una matriz de N filas por M columnas, donde cada celda puede tener un estado de 1 (activa) o 0 (inactiva).

El sistema evoluciona a lo largo de E ciclos de tiempo. Las reglas del proceso son las siguientes:

1. Inicialización algorítmica: La matriz se genera mediante una condición matemática basada en sus coordenadas.Para cada celda en la posición de la fila i y columna j (con i desde 0 hasta N-1 y j desde 0 hasta M-1), el valor inicial es 1 si la condición (i * 5 + j * 3 + 2) % 7 == 0 se cumple, y 0 en caso contrario.
2. Ciclos de evolución: Se deben simular E ciclos consecutivos. En cada ciclo, el estado de cada celda se actualiza basándose en la suma de los valores de sus 8 vecinos inmediatos (arriba, abajo, izquierda, derecha y las cuatro diagonales), utilizando condiciones de frontera toroidales (el tablero se conecta por los bordes, es decir, la fila superior se conecta con la inferior y la izquierda con la derecha).
    - La regla de transición para el siguiente estado de la celda (i, j) es: si la cantidad de vecinos activos (con valor 1) es un número par, la celda pasa a ser 1; si la cantidad de vecinos activos es impar, pasa a ser 0.(Nota: el valor de la celda propia no se cuenta dentro de sus vecinos para esta regla).

Tu tarea es escribir un programa que, dados los valores de N, M y E, ejecute la carga inicial en la matriz y los E ciclos de evolución descritos, y finalmente retorne la suma total de todos los valores de la matriz resultante.

Especificación del programa:

- Parámetros:
    1. N: Entero que indica la cantidad de filas de la matriz.
    2. M: Entero que indica la cantidad de columnas de la matriz.
    3. E: Entero que indica la cantidad de ciclos de evolución a simular.
- Valor de retorno:
    1. Un número entero que representa la suma total de los valores de todas las celdas de la matriz tras completar los E ciclos.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 5, M = 5, E = 3
    - Salida: 12

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 50, M = 50, E = 36
- Formato de respuesta esperado en el sistema: Un número

4 - En este desafío nos adentraremos en un videojuego de ingenio donde se controla un caballo de ajedrez confinado en un tablero extremadamente finito: una grilla de 2 x N casillas (solamente dos filas).

El caballo inicia en una cierta casilla inicial dada por su número de fila f y su columna c. Puede elegir moverse por donde quiera mientras respete su regla de movimiento y no se salga del tablero. En cualquier momento, se puede elegir finalizar el juego.

Las reglas del proceso y movimiento son las siguientes:

1. Inicialización algorítmica: Las dos filas de monedas de tamaño N se generan mediante condiciones matemáticas basadas en sus posiciones horizontales. Para cada columna j (desde 0 hasta N - 1):
    - El valor de la moneda en la primera fila es fila1[j] = ((j * 31 + 13) % 41) - 20.
    - El valor de la moneda en la segunda fila es fila2[j] = ((j * 37 + 17) % 41) - 20.
2. Regla de movimiento: El caballo se mueve saltando una distancia de 1 casilla en horizontal y 2 en vertical, o bien 2 casillas en horizontal y 1 en vertical. Dado que el tablero tiene únicamente 2 filas, un movimiento válido de caballo siempre implica cambiar de fila (pasar de la fila 1 a la 2, o viceversa) y desplazarse exactamente 2 casillas en horizontal (hacia la izquierda o hacia la derecha, es decir, de la columna c a c+2 o c-2), siempre y cuando la casilla de destino se encuentre dentro de los límites del tablero (0 <= columna <= N - 1).
3. Puntuación y monedas: En cada casilla hay una moneda con un valor entero (positivo, cero o negativo). Cuando el caballo pasa por una casilla, obtiene obligadamente la moneda y se suma su valor al total de puntos (incluso si el puntaje se vuelve negativo). Una vez obtenida, la moneda se consume permanentemente y no vuelve a sumarse si se visita la misma casilla de nuevo. La moneda en la casilla inicial (f, c) se obtiene inmediatamente no bien comienza el juego.

Tu tarea es escribir un programa que, dados los valores de N, la fila inicial f y la columna inicial c, genere las filas de monedas algorítmicamente y determine la máxima cantidad de puntos que se puede obtener.

Especificación del programa:

- Parámetros:
    1. N: Entero que indica la cantidad de columnas del tablero.
    2. f: Entero (1 o 2) que indica la fila inicial del caballo.
    3. c: Entero (0 a N-1) que indica la columna inicial del caballo.
- Valor de retorno:
    1. Un único número entero que representa el máximo puntaje posible que se puede acumular.

Restricciones:

- 1 <= N <= 1000
- 1 <= f <= 2
- 0 <= c <= N - 1

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 10, f = 1, c = 0
    - Salida: 12

Resultado para la entrada de prueba oculta:

Para avanzar al siguiente problema con la entrada de evaluacion oculta:

- Entrada de evaluacion: N = 500, f = 1, c = 0
- Formato de respuesta esperado en el sistema: Número